Centro Universitário de Excelência
Sistemas de Informação

Dendê Eventos API

Autor: Lucas Ferreira Lan,
César Filipe Gomes,
Rodrigo Souza Guimarães,
Italo Yan Mendes da Silva,
Valnei Sousa

Feira de Santana, 2026

Agenda

O objetivo dessa apresentação é a **refatoração da API Dendê Eventos**, melhorando a organização e manutenção do código

0. Contexto:
Quem somos nós?
Qual nosso objetivo?

1. Aplicação do padrão Builder:
Aplicação do padrão Builder na criação de eventos

2. Desativar Usuário:
A User Story da equipe "**LONDRES**".

3. Separação da aplicação em Controllers:
Estruturação da aplicação em Controllers por entidade

4.Implementação de Mappers:
Utilização de Mappers para conversão entre Model e DTO

5.Atualização do Diagrama de Classes:
Atualização do Diagrama de Classes da aplicação

6. Criação de Diagrama de Atividade:
Modelagem de Diagrama de Atividade para um endpoint

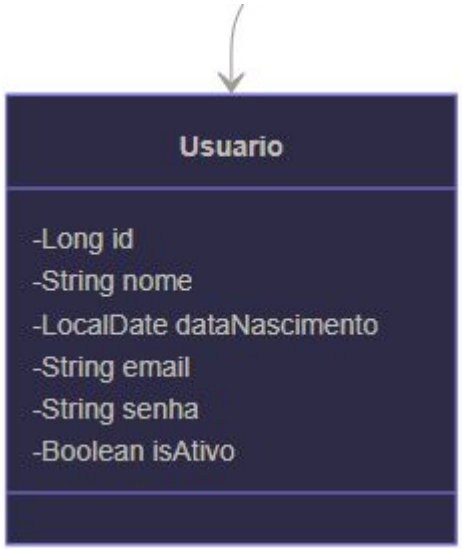
7. Considerações Finais:
Considerações finais do projeto.

Contexto do Projeto

- Somos parte da equipe da Dendê Softhouse, uma empresa baiana responsável pela plataforma Dendê Eventos.
- Como participantes do programa trainee “Dendezeiros”, atuamos diretamente no desenvolvimento Web da nova versão do sistema.
- Desta vez fazendo a refatoração e melhorias do projeto, pegando como base o que foi desenvolvido por nós anteriormente.
- Contudo o foco atualmente é a User Story: Desativar Usuário, além de outros incrementos

Padrão Builder

- O padrão **Builder** foi aplicado na criação da entidade Evento.
- O padrão auxilia na construção de objetos complexos
- Código mais legível, organizado e “Limpo”.



Padrão Builder

```
public Usuario criarUsuario(CriarUsuarioRequestDTO dto) {
    Usuario usuarioExiste = this.usuarioRepositorio.buscarUsuarioPorEmail(dto.email());

    if (usuarioExiste != null) {
        throw new RuntimeException("O email já está em uso por outro usuário");
    }

    Usuario usuario = new Usuario(dto);

    return this.usuarioRepositorio.cadastrarUsuario(usuario);
}
```

User Story: Desativar Usuário

- História do time: **LONDRES – Desativar Usuário**
- Permite alterar status do usuário para inativo
- Preserva histórico e dados no sistema
- Implementada através de endpoint da API

```java
public void desativarUsuario(long usuarioId) {
    Usuario usuarioExiste = this.usuarioRepositorio.buscarUsuarioPorId(usuarioId);

    if (Boolean.FALSE.equals(usuarioExiste.getIsAtivo())) {
        throw new IllegalStateException("Usuário já está inativo.");
    }

    usuarioExiste.setIsAtivo(false);
}
```

Separação da aplicação em Controllers

- Responsáveis por receber requisições HTTP
- Gerenciam rotas e endpoints da API
- Encaminham chamadas para a camada de serviço
- Separação entre requisição e regra de negócio

Separação da aplicação em Controllers

```java
@PatchMapping(path = "/{usuarioId}/desativar")
public ResponseEntity<String> desativarUsuario(@PathVariable(parameter = "usuarioId") long usuarioId) {

    try {
        this.usuarioService.desativarUsuario(usuarioId);

        return ResponseEntity.status(204, null);
    }catch (NotFoundException e) {
        return ResponseEntity.status(404, e.getMessage());
    } catch (Exception e) {
        return ResponseEntity.status(500, e.getMessage());
    }
}
```

DTO

- Data Transfer Object
- É o objeto usado para transferir dados entre camadas
- Auxiliando na redução do acoplamento e na proteção de entidades
- Fortalecendo a segurança, organização e controle de dados

Implementação de Mappers

- Conversão entre **Model** e **DTO**
- Padronizam transformação de dados
- Evitam exposição direta das entidades
- Facilitam reutilização de código

Implementação de Mappers

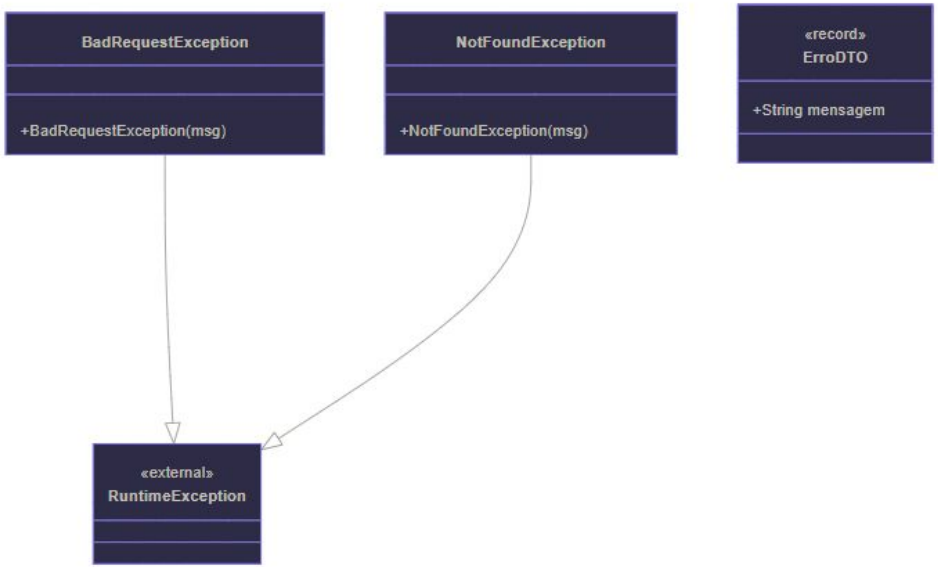
```java
public class OrganizadorMapper {

    public OrganizadorDTO toDto(Organizador organizador) {
        if (organizador == null) {
            return null;
        }

        boolean ativo = organizador.getIsAtivo() != null ? organizador.getIsAtivo() : false;

        return new OrganizadorDTO(
                organizador.getId(),
                organizador.getNome(),
                organizador.getEmail(),
                organizador.getDataNascimento(),
                calcularIdade(organizador.getDataNascimento()),
                organizador.getSexo(),
                ativo,
                organizador.getEmpresa()
        );
    }
```

Atualização do Diagrama de Classes:



Atualização do Diagrama de Classes:

Dendê Eventos API

Considerações Finais:

- Com a refatoração a organização do código teve uma grande melhoria;
- Maior padronização do projeto;
- Código mais visualmente agradável e uma legível;
- Sistema mais preparado para manutenção e escalabilidade;

Referências

- SCHILDT, Herbert. Java para iniciantes. 6. ed. Porto Alegre: Bookman, 2015. E-book. Disponível em: https://integrada.minhabiblioteca.com.br/reader/books/9788582603376. Acesso em: 12 mar. 2026.

- NITHACK, Andrey. A mágica por trás da conversão de dados: o padrão DTO explicado. Medium, 6 jul. 2023. Disponível em: https://devnit.medium.com/am%C3%A1gica-por-tr%C3%A1s-da-convers%C3%A3o-de-dados-o-padr%C3%A3odto-explicado-25f68a718b5a. Acesso em: 12 mar. 2026.